

Chapter 2: npm

Parameter Example

```
access npm publish --access=public
bin npm bin -g
edit npm edit connect
help npm help init
init npm init
install npm install
link npm link
prune npm prune
publish npm publish ./
restart npm restart
start npm start
stop npm stop
update npm update
version npm version
```

Node Package Manager (npm) provides following two main functionalities: Online repositories for node.js packages/modules which are searchable on search.nodejs.org. Command line utility to install Node.js packages, do version management and dependency management of Node.js packages.

Section 2.1: Installing packages

Introduction

Package is a term used by npm to denote tools that developers can use for their projects. This includes everything from libraries and frameworks such as jQuery and AngularJS to task runners such as Gulp.js. The packages will come in a folder typically called `node_modules`, which will also contain a `package.json` file. This file contains information regarding all the packages including any dependencies, which are additional modules needed to use a particular package.

Npm uses the command line to both install and manage packages, so users attempting to use npm should be familiar with basic commands on their operating system i.e.: traversing directories as well as being able to see the contents of directories.

Installing NPM

Note that in order to install packages, you must have NPM installed.

The recommended way to install NPM is to use one of the installers from the [Node.js download page](#). You can check to see if you already have node.js installed by running either the `npm -v` or the `npm version` command.

After installing NPM via the Node.js installer, be sure to check for updates. This is because NPM gets updated more frequently than the Node.js installer. To check for updates run the following command:

```
npm install npm@latest -g
```

How to install packages

[GoalKicker.com – Node.js Notes for Professionals 19](#)To install one or more packages use the following:

```
npm install <package-name>
```

or

```
npm i <package-name>...
```

e.g. to install lodash and express

```
npm install lodash express
```

Note: This will install the package in the directory that the command line is currently in, thus it is important to check whether the appropriate directory has been chosen

If you already have a `package.json` file in your current working directory and dependencies are defined in it, then

npm version of the npm If you want to install a specific version of a package use:

`install` will automatically resolve and install all dependencies listed in the file. You can also use the shorthand

`install` command which is: `npm i`

```
npm install <name>@<version>
```

e.g. to install version 4.11.1 of the package lodash

```
npm install lodash@4.11.1
```

If you want to install a version which matches a specific version range use:

```
npm install <name>@<version range>
```

e.g. to install a version which matches "version >= 4.10.1" and "version < 4.11.1"

of the package lodash

```
npm install lodash@">=4.10.1 <4.11.1"
```

If you want to install the latest version use:

```
npm install <name>@latest
```

The above commands will search for packages in the central npm repository at npmjs.com. If you are not looking to install from the npm registry, other options are supported, such as:

packages distributed as a tarball

```
npm install <tarball file>
```

```
npm install <tarball url>
```

packages available locally

```
npm install <local path>
```

packages available as a git repository

```
npm install <git remote url>
```

packages available on GitHub

```
npm install <username>/<repository>
```

packages available as gist (need a package.json)